ANGELES UNIVERSITY FOUNDATION
Angeles City
COLLEGE OF COMPUTER STUDIES
1st Semester, S.Y. 2026 - 2027

PDC50: Full-Stack AI Application Development
Lab Activity: ShopSmart Quality and Deployment Guided Challenge

Student Name: Sean Gabriel S. Santos	**Student Number:** 21-0207-737
Course, Year and Section: BS Information Technology 4	**Date Submitted:** November 12, 2026

SECTION 1. BASELINE ASSESSMENT

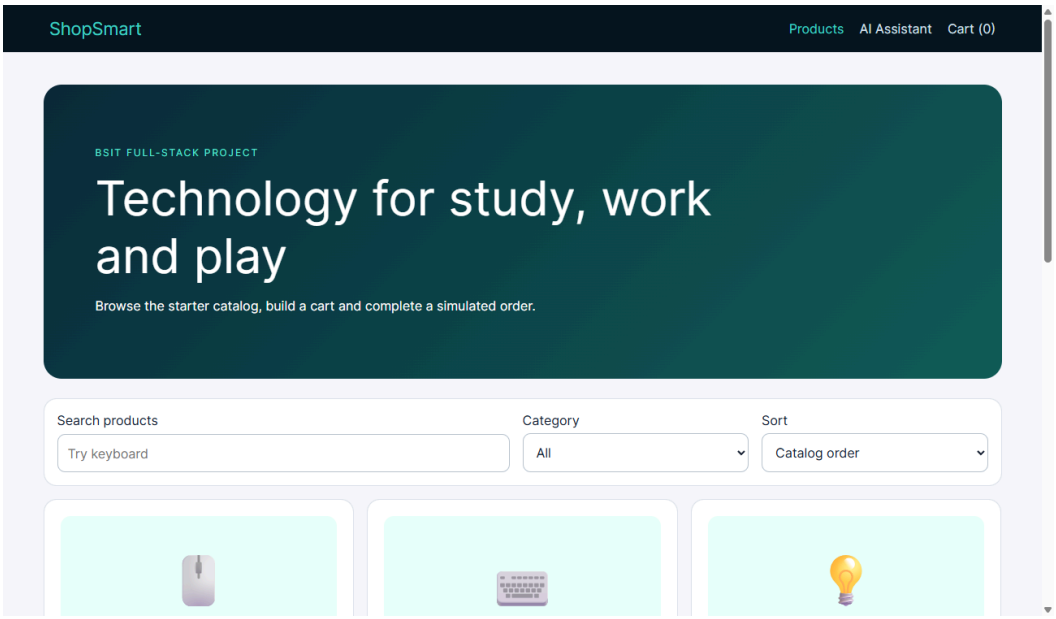
This baseline was recorded before any further fixes were made. No application code was changed during the baseline assessment. The repository was at commit 2570001 (Added product assistant) on the main branch. Quality documents, tests, and deployment notes were uncommitted local changes on that commit.

1. Environment

- The project was located in C:\ShopSmart-Starter and used Node.js v24.11.1 and npm 11.6.2.
- The PostgreSQL service (postgresql-x64-18) was already running and was not restarted. The development servers were also already running, so no additional instances were started. The frontend was accessible at http://localhost:5173, while the API was accessible at http://localhost:3000.
- The backend/.env file was present and contained the required configuration variables: DATABASE_URL, PORT, FRONTEND_URL, NODE_ENV, GEMINI_API_KEY, GEMINI_MODEL, and GEMINI_TIMEOUT_MS. The values were not copied or recorded. Similarly, the frontend/.env file was present, with VITE_API_URL configured. Its value was not recorded.
- The GEMINI_API_KEY was confirmed to be set on the server, but the key itself will not be recorded here.
- The command npm run db:seed was not executed during the baseline assessment.

2. Home Page and API Health

- The home page at http://localhost:5173/ loaded successfully and displayed six products. The shopping cart initially contained zero items.
- The available product categories were All, Accessories, Study, Video, and Audio. The available sorting options were Catalog Order, Price: Low to High, and Price: High to Low.



- The six products displayed on the home page matched the records returned by GET /api/products.

ID	Product Name	Category	Price	Stock
1	Wireless Mouse	Accessories	₱650.00	9
2	Compact Mechanical Keyboard	Accessories	₱1,850.00	6
3	USB-C Study Lamp	Study	₱900.00	8
4	1080p Webcam	Video	₱1,450.00	6
5	Laptop Stand	Study	₱1,200.00	9
6	USB Headset	Audio	₱1,350.00	7

- All six products were confirmed to be active.
- The API health endpoint, GET http://localhost:3000/api/health, returned HTTP status 200 with the response body {"status":"ok"}. It also returned the request

identifier a8facd5a-be4b-44f3-9df5-589cd02f7238 through the X-Request-ID response header. Opening the same URL directly in the browser displayed the same response.

- The database contained six products, six orders, and six order items. All six orders had a status of PENDING.
- The recorded order totals were as follows:
    - Order #2: ₱650
    - Order #6: ₱900
    - Order #7: ₱3,700
    - Order #8: ₱900
    - Order #9: ₱650
    - Order #10: ₱650
- The Wireless Mouse had a remaining stock of nine units because Order #10 had previously purchased one mouse on the same date.

3. Automated Quality Checks

The command npm run check was executed from C:\ShopSmart-Starter. The command completed successfully with an exit code of 0. The total execution time was approximately 59 seconds. The following checks were performed:

Check	Workspace	Result
eslint .	Frontend	Passed with no findings
eslint .	Backend	Passed with no findings
vitest run	Frontend	Passed: 1 test file and 4 tests
node --test	Backend	Passed: 32 tests, 0 failures
vite build	Frontend	Passed

The frontend unit tests covered frontend/src/utils/cart.test.js, with all four tests passing. The backend test results were distributed across the following files:

Test File	Number of Tests
backend/test/assistant.test.js	22
backend/test/app.test.js	5

backend/test/validation.test.js	3
backend/test/seed-guard.test.js	2
Total	**32**

The production build also completed successfully using Vite 7.3.6. The generated files included frontend/dist/index.html, frontend/dist/assets/index-BcvfYfWg.css, and frontend/dist/assets/index-BAUtdohV.js. No automated check failed during execution. Therefore, no failing workspace or test case was reproduced during the baseline assessment.

However, the backend logs displayed two error messages: a JSON parsing error involving {bad and an error containing the text secret-do-not-leak. These messages were generated by two passing tests: malformed JSON returns a generic client error and a request id and an unexpected server failure stays generic and keeps the trace id. Both tests were marked as passed. These log messages were expected outputs from the test scenarios and did not indicate failures in the automated test suite.

4. Implemented Features and Current Limitations

The following table summarizes the features that were present or absent in the application at the time of the baseline assessment.

Feature	Implementation Status and Description
Authentication	Not implemented. There is no login, account, session, or password-related functionality in the frontend or API. Existing product and order comments identify authentication as a future improvement.
User Accounts	Not implemented.
Live AI	Implemented at the code level. The backend service backend/src/services/aiProvider.js communicates with Gemini, and POST /api/assistant/questions uses generateRecommendation. The GEMINI_API_KEY is configured on the server. However, a successful live model response was not captured during this baseline assessment.

Product Browsing	Implemented. Users can view products through the home page and the GET /api/products endpoint.
Product Search	Implemented. Users can search for products through the search box, which uses the GET /api/products?search= endpoint.
Category Filtering and Price Sorting	Implemented. Users can filter products by category and sort them by price on the home page.
Product Details	Implemented. Individual product information is accessible through /products/:productId.
Shopping Cart	Implemented. Cart data is stored in memory and is cleared when the page is refreshed.
Simulated Checkout	Implemented. Users can proceed through the checkout process, but no actual payment is collected.
Stock Validation	Implemented. Product stock is validated in the shopping cart and through the POST /api/orders endpoint.
Product Assistant	Implemented. Available at /assistant, with catalog-based rules and a Gemini client integration.
API Health Monitoring	Implemented. The /api/health endpoint checks the API and its PostgreSQL database connection.
Order History	Not implemented. There is no API route for retrieving a list of previous orders.
Product Administration	Not implemented.
Real Payment Processing	Not implemented.

5. Frontend and Backend Integration

The frontend communicates with the backend through frontend/src/services/api.js. Product information, including prices and stock quantities, is retrieved from the Express API, which uses PostgreSQL as its data source.

The product assistant also obtains its information through the backend integration. The Gemini API key remains on the server and is not directly exposed to the frontend.

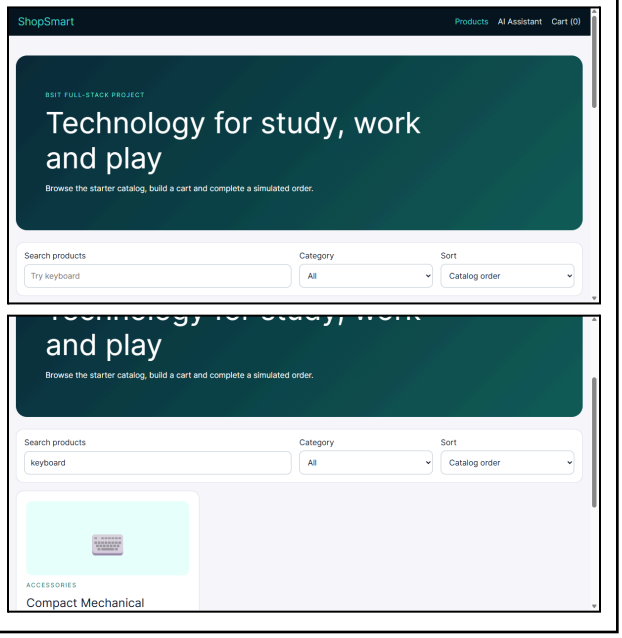
This approach keeps the frontend dependent on the backend for product data and assistant-related information, rather than relying on hardcoded product values or exposing sensitive API credentials in the client application.

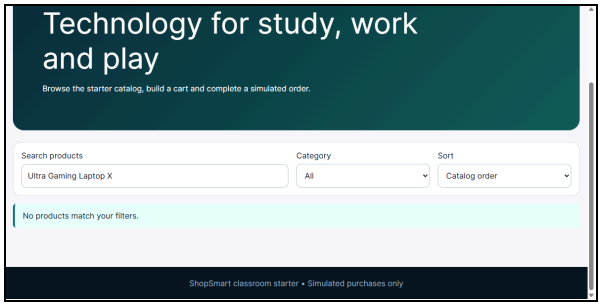
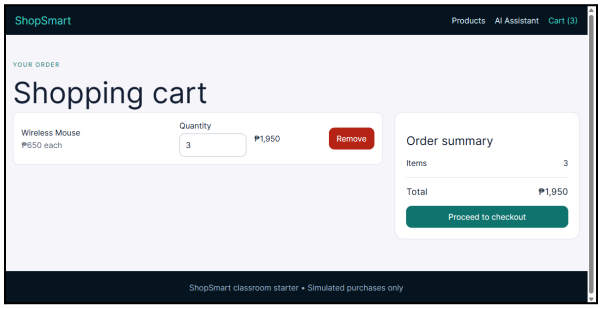
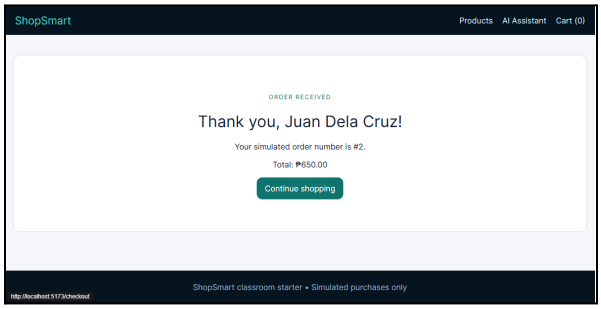
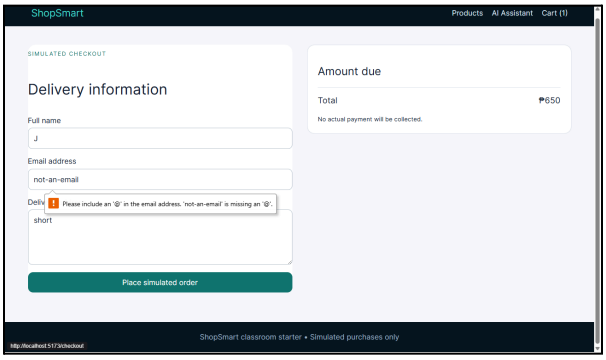
Overall, the baseline confirmed that the core product browsing, searching, filtering, shopping cart, simulated checkout, stock validation, and product assistant features were implemented. The application also passed all available automated checks, including linting, unit tests, backend tests, and production build verification.

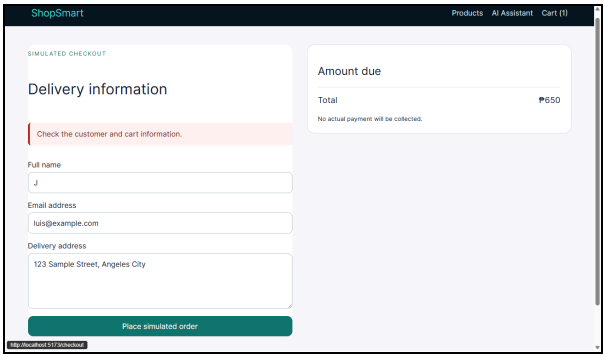
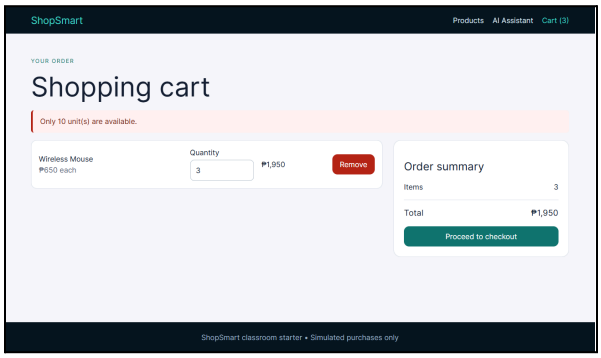
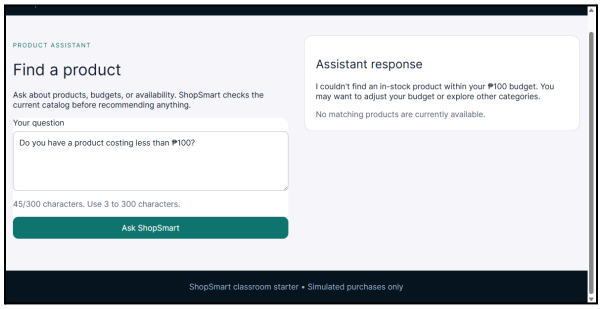
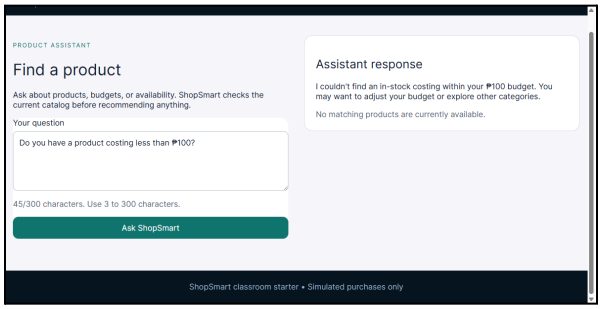
However, authentication, user accounts, order history, product administration, and real payment processing remained unimplemented. Additionally, although the Gemini integration was present in the codebase and its API key was configured, successful live AI functionality still required separate verification.

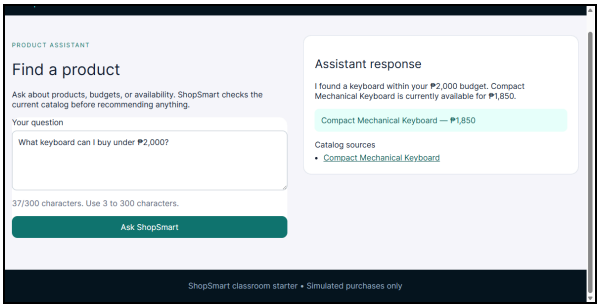
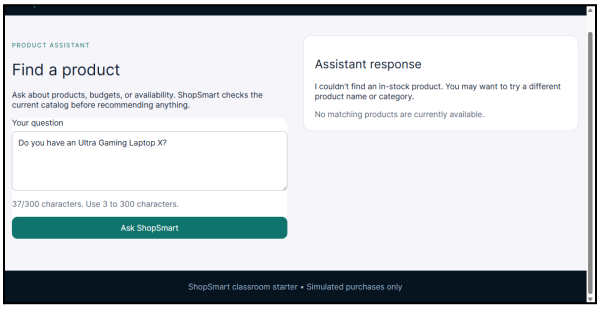
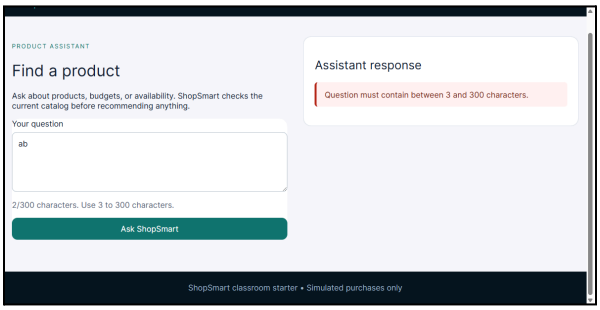
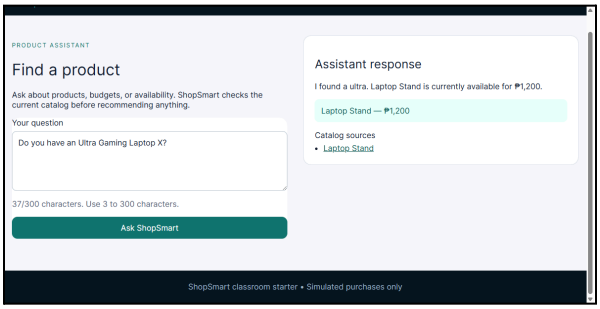
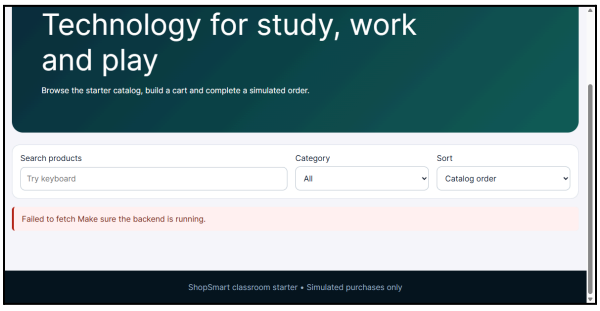
SECTION 2. CUSTOMER JOURNEY TESTING & VALIDATION

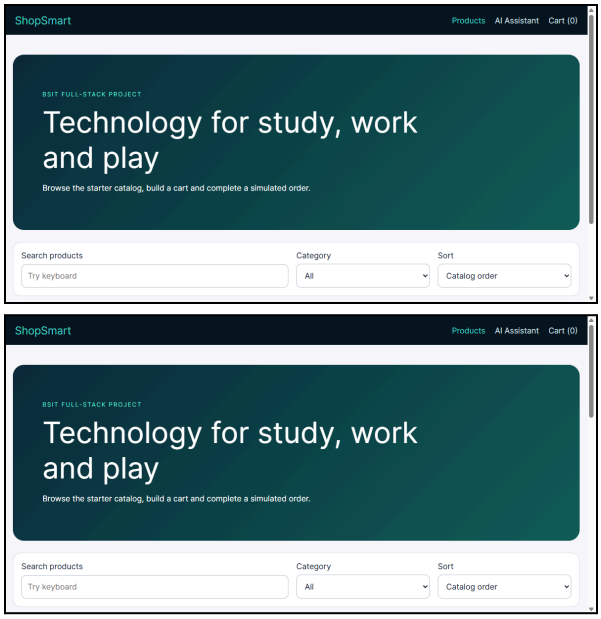
The ShopSmart application is tested across its core customer journey to verify functionality, identify defects, and document results with supporting evidence.

Case	Expected Behavior	Evidence to Retain
Browse and Search	Products load; search returns relevant results and a useful no results state	

		
Cart	Add, change quantity, remove, and refresh as your team's design allows. Totals remain correct.	
Checkout	Valid fictional details create one simulated order; invalid details are refused.	 

		
Stock	An unavailable product cannot be purchased beyond the applicable stock rule.	
Assistant	A supported catalog question gives correct products and sources; no match is honest.	 

		   
Failure	An API or AI provider failure ends loading and shows a useful message or fallback.	

SECTION 3. DEFECT DIAGNOSIS & FIXES

Field	Description
Starting State	The catalog contains six active products. None is named “key.” The Compact Mechanical Keyboard (Product ID: 2) is available at ₱1,850.
Test Action	Send a POST /api/assistant/questions request with the question: {"question":"Do you have a key?"}
Actual Result	The API returned HTTP 200 and recommended the Compact Mechanical Keyboard: “I found a product. Compact Mechanical Keyboard is currently available for ₱1,850.” The response also included Product ID 2 and the link /products/2.
Expected Result	The assistant should recognize that no product matches the term “key” and return an honest no-match response instead of recommending the Compact Mechanical Keyboard.

SECTION 4. DEPLOYMENT ARCHITECTURE & PLAN

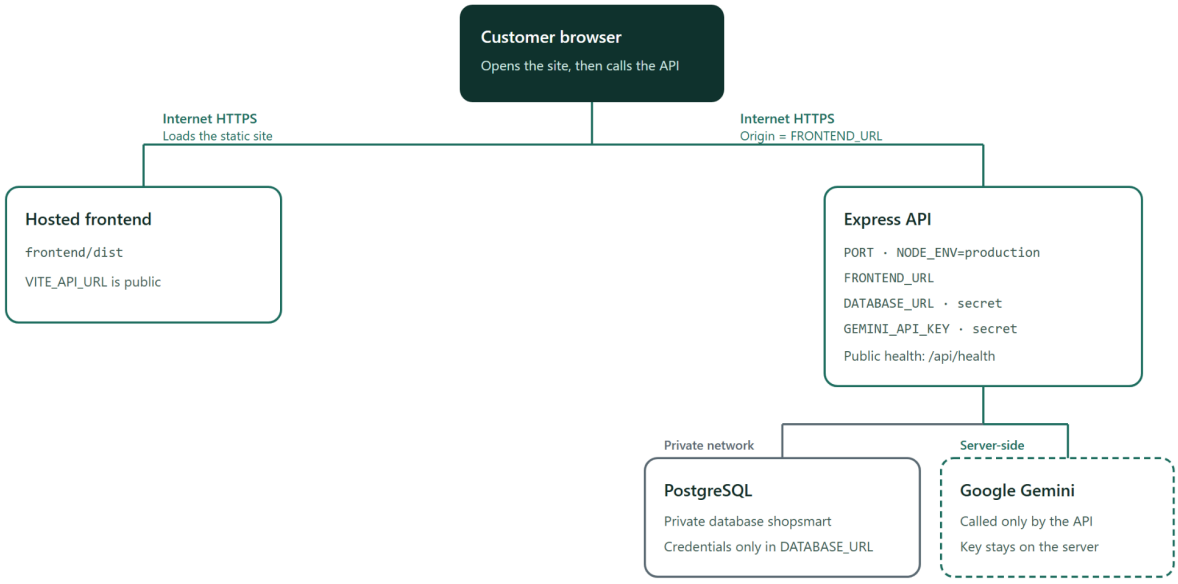
ShopSmart follows a client-server architecture in which the browser loads the hosted frontend and communicates directly with the Express API through HTTPS requests. The API handles application logic, database operations, and communication with the Google Gemini service.

PostgreSQL and the Gemini API key remain on the server side and are never exposed to the browser.

Deployment Status: No public hosting provider or staging environment has been approved or configured. Therefore, the architecture below presents the proposed deployment structure and configuration requirements, not an active production deployment.

1. Deployment Architecture

The diagram illustrates how the frontend, backend, database, and external AI service communicate during deployment.



2. Frontend Deployment

The React application is built using Vite. Running npm run build generates the production-ready static files in frontend/dist. This directory is the deployment output that must be uploaded to or served by the selected frontend hosting provider.

The frontend communicates with the Express API using the VITE_API_URL environment variable.

- Local development: http://localhost:3000/api
- Proposed production configuration: https://<approved-api-host>/api

Before building for production, the hosting environment must be configured with the approved API address through VITE_API_URL. The frontend host must also be configured to serve index.html as the fallback for application routes such as /products/1, /cart, /checkout, and /assistant. This ensures that direct navigation and page refreshes work correctly.

Security consideration: Vite exposes variables prefixed with VITE_ in the generated browser bundle. Therefore, VITE_API_URL must contain only the public API address. Database credentials, provider keys, and other confidential values must never be stored in frontend environment variables.

3. Backend API Deployment

The backend is an Express application responsible for processing frontend requests, communicating with PostgreSQL, and handling requests to the Gemini API.

The production server is started using:

```
npm run start --workspace backend
```

This executes node src/server.js and listens on the port specified by the PORT environment variable.

The API health endpoint is used to verify whether the backend is running:

- Local endpoint: http://localhost:3000/api/health
- Proposed production endpoint: https://<approved-api-host>/api/health
- Expected response: {"status":"ok"}

The backend must run in production mode using NODE_ENV=production. Its environment must also contain the required configuration values, including PORT, FRONTEND_URL, DATABASE_URL, and GEMINI_API_KEY.

4. Database Deployment

ShopSmart uses PostgreSQL to store product, order, and order-item records. The database must remain accessible only to the backend through a private database connection. The browser must never connect directly to PostgreSQL. The current local environment uses:

- Database system: PostgreSQL
- Database name: shopsmart
- Local host: localhost
- Port: 5432

The backend obtains its database connection through the DATABASE_URL environment variable. During local development, this value is stored in backend/.env, which is excluded from version control.

For production, the approved hosting environment must provide the database connection string securely through its environment-variable or secret-management settings. Database credentials must not be committed to the repository or exposed through the frontend.

5. CORS Configuration

Cross-Origin Resource Sharing (CORS) controls which browser origins are allowed to send requests to the backend API.

ShopSmart uses the FRONTEND_URL environment variable to identify the permitted frontend origin.

- Local development: http://localhost:5173
- Proposed production configuration: https://<approved-frontend-host>

The production value must contain only the frontend origin, without an additional path. The backend requires this variable in production and rejects the wildcard value (*) to prevent unrestricted cross-origin access.

Once hosting is approved, FRONTEND_URL must match the actual deployed frontend origin.

6. Gemini API Configuration

The product assistant communicates with Google Gemini through the backend. The browser sends requests to the Express API, which handles communication with the external AI service. The following environment variables are used:

- GEMINI_API_KEY = authenticates requests to the Gemini API.
- GEMINI_MODEL – identifies the configured Gemini model.
- GEMINI_TIMEOUT_MS – defines the request timeout.

The API key is stored only in the backend environment and is sent to Gemini through the x-goog-api-key request header. It must never be included in frontend code, exposed through a VITE_ variable, or committed to the repository. Environment files are excluded from Git through .gitignore.

7. Database Migration and Seeding Procedure

Database migrations must be applied before the backend serves a newly deployed database. ShopSmart uses Prisma 6 to manage database schema changes. The following commands have different purposes:

Command	**Purpose**
npm run db:migrate:deploy	Applies committed Prisma migrations to the deployment database.
npm run db:migrate	Runs prisma migrate dev for local development and migration authoring.
npm run db:seed	Inserts sample records into the database.

For production deployment, npm run db:migrate:deploy must be executed before the API begins serving requests against the updated database.

Important data protection measure: Do not run npm run db:seed against a database containing existing orders or other records that must be preserved. When SHOPSMART_ALLOW_SEED=yes is enabled, the seed process deletes existing

products, orders, and order items before inserting sample data. Seeding must therefore be restricted to controlled development or disposable test environments.

8. Staging Environment and Access Control

No staging environment has been created because public hosting has not yet been approved. If an instructor authorizes staging deployment, the team should configure access so that the environment remains private to the class.

The staging environment must use fictional customer information for testing, including names, email addresses, and delivery addresses. The checkout process is simulated and must not collect or process actual payment information.

Before staging is considered ready, the team should verify:

- Frontend accessibility and API connectivity.
- Successful backend health checks.
- Database connectivity and migration status.
- Product browsing and simulated checkout.
- Product assistant behavior and external API connectivity.
- Proper handling of environment variables and confidential credentials.
- Appropriate CORS restrictions and access controls.

9. Deployment Readiness Summary

The following table distinguishes the current local configuration from the proposed deployment requirements.

Component	Current Status	Deployment Requirement
Frontend	Runs locally through Vite	Build and host frontend/dist as a static website.
Backend API	Runs locally through Express	Deploy the API and configure its production environment variables.
Postgre SQL	Local database named shopsmart	Use a secured database connection accessible to the backend.

Gemini API	Integrated through the backend service	Configure the provider key securely in the backend environment.
CORS	Configured for the local frontend origin	Set FRONTEND_URL to the approved production frontend origin.
Database Migrations	Prisma migrations are available	Apply committed migrations using npm run db:migrate:deploy.
Staging	Not yet created	Deploy only after approval and restrict access to the class.

Overall Deployment Status: ShopSmart has a defined deployment architecture and documented configuration requirements. However, public hosting and staging deployment remain pending approval. The application should therefore be considered a locally verified release candidate rather than a publicly deployed production system.

SECTION 5. RELEASE CANDIDATE VERIFICATION

This section presents the final verification of the ShopSmart 1.0.0 local release candidate conducted on November 12, 2026. The verification assessed whether the application could be built, executed, and used for its core functions while checking the reliability of its backend, database operations, product assistant, and security configuration.

The verification was performed in the local development environment because no public hosting provider or staging environment had been approved. Therefore, the results establish the readiness of the local release candidate and do not represent a completed production or staging deployment.

1. Release Candidate Information

The verification was conducted using the existing local application without starting a second development server or modifying the existing database through sample-data seeding.

Item	**Recorded Configuration**
Project Version	ShopSmart 1.0.0

Verification Date and Time	November 12, 2026, 12:31 AM–12:40 AM (UTC+8)
Release Candidate	Local
Frontend URL	http://localhost:5173
Backend API URL	http://localhost:3000
Frontend API Configuration	VITE_API_URL=http://localhost:3000/api
Database	PostgreSQL – shopsmart, schema public
Database Host and Port	localhost:5432
Git Branch	main
Git Commit	2570001d9fb5591ed0910b7f6bff01d4635150ff (Added product assistant)
Working Tree	Quality notes, deployment notes, and earlier fixes remained uncommitted and were preserved.

Before verification, the existing development server was confirmed to be running. The Prisma migration status command reported that the database schema was up to date, with no migration applied during this check. The database seeding command was not executed to avoid deleting or replacing existing records.

2. Build and Automated Test Verification

The automated quality checks were executed from the project root directory using npm run check. The command completed successfully with an exit code of 0 in approximately 28 seconds.

Verification Check	**Command**	**Actual Result**	**Status**
Frontend Linting	eslint .	No linting findings were reported.	PASS
Backend Linting	eslint .	No linting findings were reported.	PASS
Frontend Tests	vitest run	Four tests passed in src/utils/cart.test.js.	PASS

Backend Tests	node --test	34 tests passed with no failures.	PASS
Production Build	vite build	Vite 7.3.6 successfully generated the production files in frontend/dist.	PASS
Complete Quality Check	npm run check	The complete command finished with exit code 0.	PASS

The 34 backend tests covered the following test files:

- assistant.test.js – 24 tests
- app.test.js – 5 tests
- validation.test.js – 3 tests
- seed-guard.test.js – 2 tests

During execution, the logs included a JSON parsing error for {bad and an error containing secret-do-not-leak. These messages were generated by passing tests that verify generic error responses for malformed JSON and unexpected server failures. They did not indicate failed tests. Prisma also displayed its existing warning regarding the deprecated package.json#prisma configuration. Overall, the frontend and backend passed their linting checks, all 38 automated tests passed, and the production frontend build completed successfully.

3. Application Smoke Test Verification

After the automated checks, the application underwent a final smoke test to verify that its primary functions remained operational in the local environment. The verification focused on application availability, product retrieval, cart calculations, simulated checkout, database persistence, and product assistant behavior.

Test Area	**Expected Result**	**Actual Result**	**Status**
API Health	The backend should respond successfully and return the expected health status.	GET /api/health returned HTTP 200 with {"status":"ok"}. The response included a request ID and the	PASS

		permitted frontend origin in the CORS header.	
Product Retrieval	Active products should load correctly from the API and appear in the storefront.	GET /api/products returned HTTP 200 with six active products. Their names, prices, and stock quantities matched the storefront.	PASS
Shopping Cart	Product quantities and cart totals should reflect the selected quantities and catalog prices.	Adding a Wireless Mouse changed the cart count from 0 to 1. Increasing its quantity to 2 displayed Cart (2), with a line total and cart total of ₱1,300.	PASS
Simulated Checkout	A valid fictional order should be accepted, stored, and followed by a cleared cart.	The application displayed confirmation for order #12 with PENDING status and a total of ₱1,300.00. The cart returned to zero items, and POST /api/orders returned HTTP 201.	PASS
Database Persistence	The order should use the database product price, update stock correctly, and preserve existing records.	Order #12 was stored with two units of Wireless Mouse at ₱650 each. Stock decreased from 7 to 5. Existing orders #2, #6, #7, #8, #9, #10, and #11 remained unchanged, and all six products remained active.	PASS
Product Assistant	The assistant should recommend a relevant, available catalog product with accurate information.	The question, “Which keyboard is below ₱2,000?” returned a recommendation for the Compact Mechanical Keyboard at ₱1,850. The product link opened /products/2, where the displayed price and stock were ₱1,850.00 and six units, respectively.	PASS

Assistant Provider Failure	The assistant should handle an external provider failure without exposing sensitive information or returning an uncontrolled error.	The Gemini provider returned HTTP 503. The backend logged a fallback to catalog rules and returned HTTP 200 after approximately 4,408 milliseconds. The page displayed a catalog-based answer without exposing the provider key.	PASS

4. Product Inventory Before Checkout

The following table records the six active products and their stock quantities before the simulated checkout was performed.

Product ID	Product Name	Price	Available Stock
1	Wireless Mouse	₱650.00	7
2	Compact Mechanical Keyboard	₱1,850.00	6
3	USB-C Study Lamp	₱900.00	8
4	1080p Webcam	₱1,450.00	6
5	Laptop Stand	₱1,200.00	9
6	USB Headset	₱1,350.00	7

5. Checkout Data and Database Verification

The checkout test used fictional customer information to verify order creation without processing an actual payment.

Field	Test Value
Customer Name	Carmen Flores
Email Address	carmen@example.com
Delivery Address	88 Sample Street, Angeles City

Product ID	1 – Wireless Mouse
Quantity	2
Database Unit Price	₱650.00
Expected Order Total	₱1,300.00

The application successfully created order #12 with a PENDING status. The database stored the unit price of ₱650.00, matching the product's catalog price, and calculated the total as ₱1,300.00. The Wireless Mouse stock decreased from seven to five units.

The existing orders and the stock quantities of the other five products remained unchanged. These results confirmed that the checkout operation used the database product price, persisted the new order, updated the corresponding stock, and preserved existing records.

The checkout process remained simulated. No actual payment information was collected or processed.

6. Product Assistant and Failure Handling

The product assistant was tested using a question about a keyboard within a specified budget. It returned the Compact Mechanical Keyboard at ₱1,850, and the provided product link directed the user to the correct product page.

A separate verification was performed when the Gemini provider returned HTTP 503. Instead of terminating the request with an uncontrolled error, the backend used its catalog-based fallback and returned a valid response. This demonstrated that the assistant could still provide catalog information when the external provider was unavailable.

The automated tests also passed for missing provider credentials, HTTP errors, timeouts, and malformed provider responses. However, a successful live Gemini-generated response was not established during this verification because the provider returned HTTP 503.

7. Security and Privacy Verification

Security checks were performed to confirm that confidential environment variables were not included in the frontend build or committed as private credentials. The verification also checked whether demonstration data and screenshots contained sensitive information.

Security Check	Expected Result	Actual Result	Status
Environment Files	Private .env files should not be committed to the repository.	git ls-files returned only backend/.env.example and frontend/.env.example. The .gitignore file excludes .env and .env.local.	PASS
Database Credentials	The database connection string should remain accessible only to the backend.	DATABASE_URL was configured in backend/.env and was not found in the frontend source or production build. Searches of frontend/dist found no DATABASE_URL or postgresql:// values.	PASS
Gemini API Key	The provider key should remain on the backend and must not appear in frontend files.	GEMINI_API_KEY was configured in backend/.env, while the example configuration left the value empty. Searches of frontend/dist found no GEMINI_API_KEY, AIza, or x-goog-api-key strings.	PASS
Frontend Bundle	Only the public API address should be exposed through the frontend configuration.	The production JavaScript bundle contained http://localhost:3000/api as its API address.	PASS
Demonstration Data	Testing should use fictional customer information.	The checkout test used fictional information for Carmen Flores.	PASS
Screenshots	Evidence should not expose credentials or confidential configuration values.	The saved screenshots showed the storefront, cart, checkout, order confirmation, assistant response, and keyboard page without displaying database credentials or the provider key.	PASS

One additional observation was recorded: README.md, backend/.env.example, and backend/test/assistant.test.js still contain the starter project's local classroom connection string. This was already present in the project and is not the private credential stored in backend/.env. The string is not reproduced in this report and must not be reused outside the intended classroom database environment.

8. Staging Verification

Staging deployment was not performed. No approved hosted frontend, backend API, or database was available during the verification period. The team did not create or modify any external hosting environment. All release checks were therefore conducted using the local frontend and backend URLs.

Once a private staging environment is approved, the same essential release checks should be repeated using the actual deployed URLs. The verification record should include the deployment date, frontend and API addresses, test results, and any remaining issues.

9. Remaining Limitations

The verification identified several limitations that should be considered when evaluating the release candidate. These findings did not prevent the local application from passing the recorded release checks, but they should remain documented for future improvements.

Issue	Potential Impact	Release Impact	Temporary Measure or Limitation
Live Gemini response unavailable during verification	The external provider returned HTTP 503, preventing confirmation of a successful live AI-generated response during this run.	Does not block the local candidate because the catalog fallback worked.	Continue using the catalog-based fallback when the provider is unavailable. Repeat a live assistant check when Gemini is accessible.

F-07: Missing CHECK (stock >= 0) database constraint	A direct SQL update could store a negative stock value. The checkout process still rejects insufficient stock within its transaction.	Does not block the verified checkout flow.	Avoid modifying product stock through direct SQL operations.
F-06: Maximum quantity is 20	The quantity ceiling was not reached during this verification because no product had more than nine available units.	Does not block the tested customer journey.	Retain the existing quantity validation and test the upper limit when suitable inventory is available.
Classroom connection string in tracked files	A local classroom connection string remains present in the README, environment example, and assistant test file.	Does not expose the private .env credential, but requires attention before broader deployment.	Do not reuse the classroom password outside its intended environment. Review tracked configuration examples before deployment.
No approved hosted environment	Public deployment and remote staging behavior could not be verified.	Limits the verification to the local environment.	Keep the release candidate local until hosting and database deployment are approved.

10. Release Decision

Based on the completed verification, ShopSmart 1.0.0 is considered ready as a local release candidate with documented limitations.

The automated quality checks completed successfully, including frontend and backend linting, four frontend tests, 34 backend tests, and the production build. The API health check returned the expected response, and the storefront displayed all six active products with matching catalog information.

The simulated checkout successfully created order #12 using the database unit price, calculated the correct total of ₱1,300.00, and reduced the Wireless Mouse stock from seven to five units. Existing orders and the remaining product records were preserved.

The product assistant also returned an accurate catalog recommendation when the Gemini provider was unavailable by using its catalog-based fallback.

Security checks confirmed that the private database URL and Gemini API key were not exposed in the production frontend bundle. Testing also used fictional customer information and preserved the existing database records.

The remaining limitations, particularly the unavailable live Gemini response, missing database stock constraint, and absence of an approved staging environment, should be addressed or considered before a broader deployment.

This decision applies only to the verified local release candidate. It does not indicate that ShopSmart has been deployed to a public production or staging environment.

11. Verification Evidence

The following screenshots were retained to support the recorded release verification results.

Evidence File	Description
release-home.png	Storefront displaying the active product catalog.
release-cart.png	Shopping cart displaying the selected product and calculated total.
release-checkout.png	Checkout form using fictional customer information.
release-order.png	Successful order confirmation for order #12.
release-assistant.png	Product assistant response and catalog-based recommendation.
release-keyboard.png	Compact Mechanical Keyboard product page showing its price and available stock.

These screenshots, together with the automated test results and database verification records, provide supporting evidence for the local release candidate assessment.

SECTION 6. TEST RECORDS

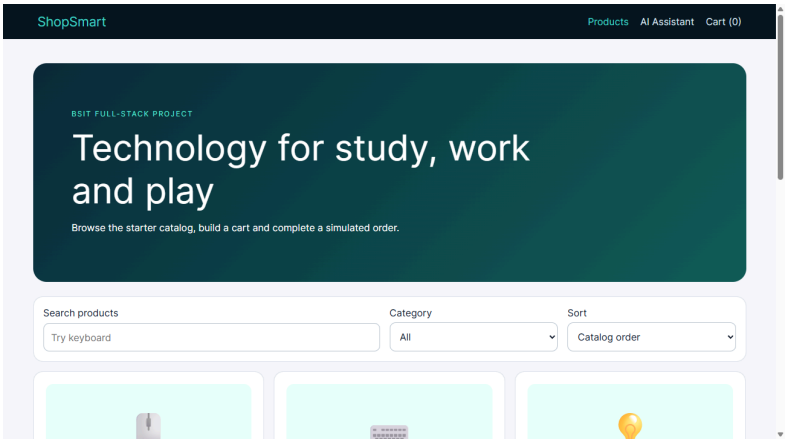
The local release candidate was subjected to five smoke tests on November 12, 2026, to verify the core ShopSmart functions before release. Testing covered API availability, product catalog loading, cart calculations, simulated checkout, and AI assistant fallback behavior. Since no hosted staging environment had been approved, all tests were conducted in the local development environment.

TC-61: API Health

Field	**What to record**
Case and starting state	API is running on port 3000, and the PostgreSQL database shopsmart is reachable.
Action and test data	Send GET http://localhost:3000/api/health.
Expected result	The API returns HTTP 200 with {"status":"ok"}, does not expose the database connection string, and allows the local frontend origin.
Observed result and evidence	The API returned HTTP 200 with {"status":"ok"}. Request ID: cbf7ed2d-c0af-4e85-b1bf-34a6c6a54ecb. The Access-Control-Allow-Origin header returned http://localhost:5173.
Decision	Pass

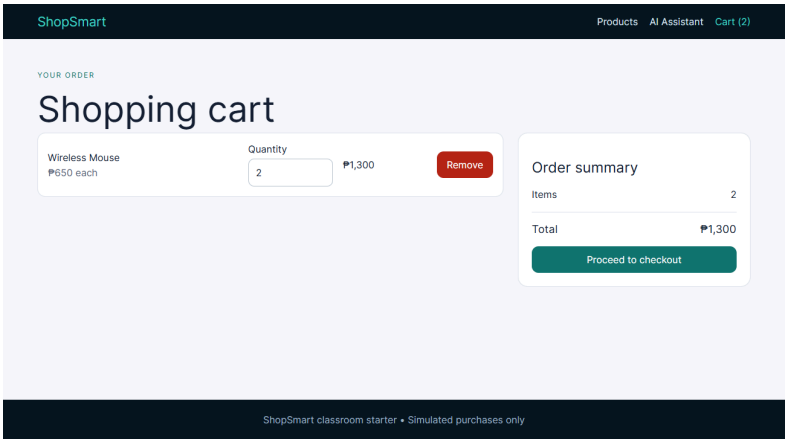
TC-62: Load the Current Catalog

Field	**What to record**
Case and starting state	Frontend and API are running. Cart is empty (0). Available products: Mouse (7), Keyboard (6), Lamp (8), Webcam (6), Stand (9), and Headset (7).
Action and test data	Open http://localhost:5173/ and wait for the product cards to load. Compare the displayed products with the response from GET http://localhost:3000/api/products.

Expected result	The homepage displays the same six active products, prices, and stock quantities returned by the API.
Observed result and evidence	Both the homepage and API displayed the same six products with matching prices and stock quantities. Screenshot: 
Decision	Pass

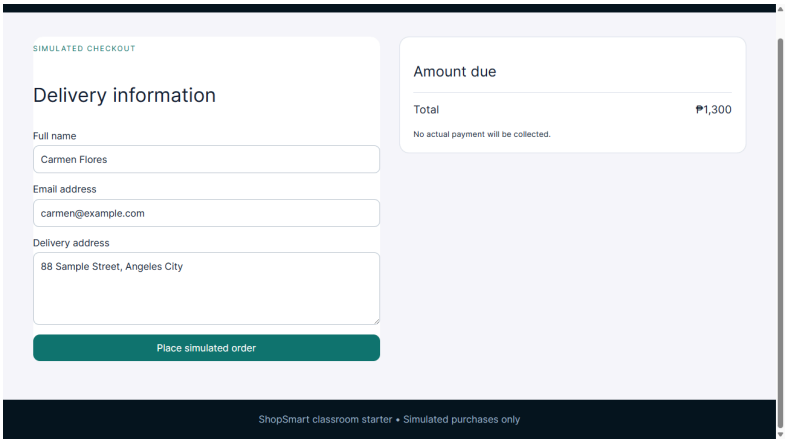
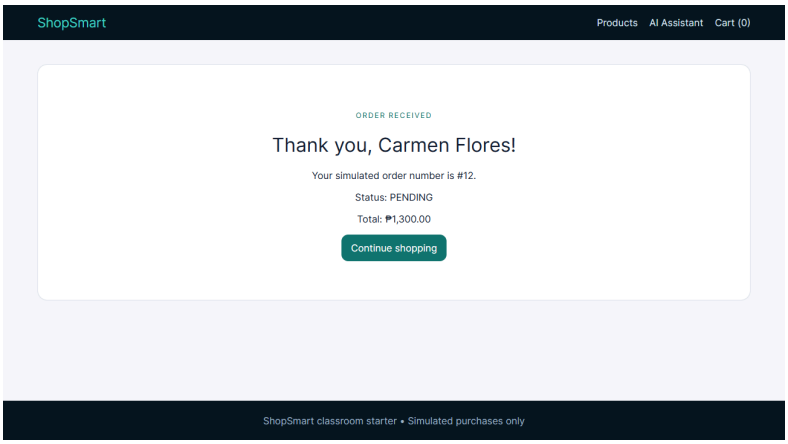
TC-63: Cart Quantity and Total

Field	**What to record**
Case and starting state	Homepage is loaded. Cart is empty (0). Wireless Mouse costs ₱650.00, with seven available units.
Action and test data	Click Add to cart on Wireless Mouse. Click Cart (1). On the /cart page, change the Quantity from 1 to 2.
Expected result	The cart displays two items, a unit price of ₱650.00, a line total of ₱1,300.00, and an overall total of ₱1,300.00.
Observed result and evidence	The cart correctly displayed the quantity of two, a line total of ₱1,300.00, and an overall total of ₱1,300.00.

	Screenshot: 
Decision	Pass

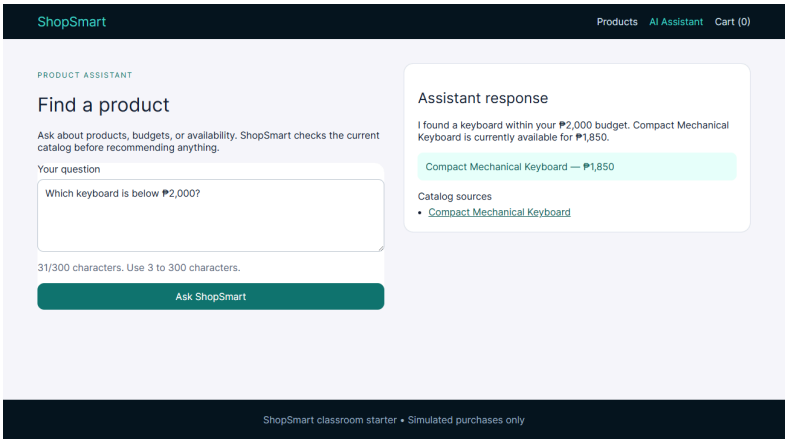
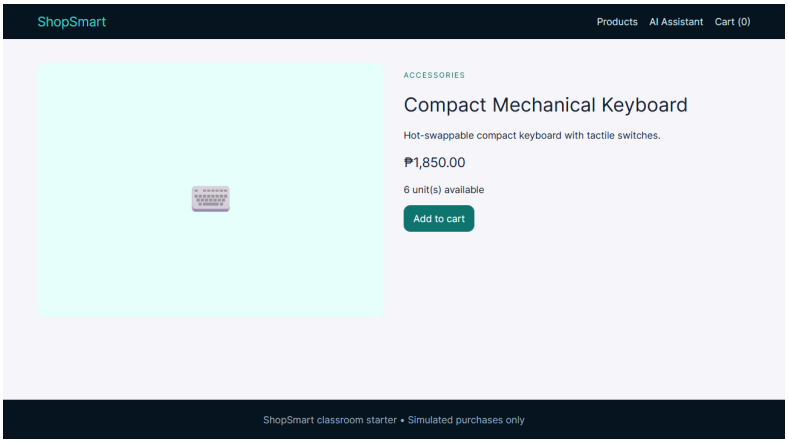
TC-64: Simulated Checkout with Two Mice

Field	**What to record**
Case and starting state	Cart contains two Wireless Mice worth ₱1,300.00. Mouse stock is seven units. Orders #2 and #6–#11 already exist in the database.
Action and test data	Click Proceed to checkout. Enter the following fictional customer details: Name: Carmen Flores; Email: carmen@example.com; Address: 88 Sample Street, Angeles City. Click Place simulated order. Do not run npm run db:seed.
Expected result	The system creates one PENDING order using the database price, clears the cart after successful checkout, reduces mouse stock from seven to five, and preserves all existing orders.
Observed result and evidence	Order #12 was successfully created with a PENDING status and a total of ₱1,300.00. POST /api/orders returned HTTP 201. The cart was cleared, mouse stock decreased from seven to five, and existing orders #2 and #6–#11 remained unchanged.

	Screenshots:  
Decision	Pass

TC-65: Assistant Recommendation When the Provider Fails

Field	**What to record**
Case and starting state	Cart is empty (0) after Order #12. Compact Mechanical Keyboard costs ₱1,850.00, with six available units. The provider API key remains configured on the backend.
Action and test data	Open AI Assistant. Enter Which keyboard is below ₱2,000?. Click Ask ShopSmart. Select Compact Mechanical Keyboard — ₱1,850 from the recommendation.

Expected result	The assistant recommends the Compact Mechanical Keyboard at ₱1,850.00 and opens /products/2 when selected. If the external AI provider fails, the system returns a catalog-based recommendation without exposing the provider API key.
Observed result and evidence	The provider returned a 503 error. The system successfully activated the catalog-based fallback and recommended the Compact Mechanical Keyboard at ₱1,850.00. The API returned HTTP 200 in 4,408 ms. The /products/2 page displayed the correct price and six available units. API log: falling back to catalog rules (PROVIDER_ERROR 503). Screenshots:  
Decision	Pass. The catalog fallback worked as expected. However, successful live Gemini integration remains blocked under the earlier TC-47 result.